Final Project Report

Implement Client Script & UI Policy (Incident)

Executive Report Metadata

• **Project Title:** Implement Client Script & UI Policy (Incident)

• **Platform:** ServiceNow Incident Management

• **Project Lead:** Rahul

• **UI Policy Owner:** Dinesh Kumar

• **Client Script Owner:** Abisheknathan

• **UAT & Final Report:** Dakshanraj

• **Status:** Completed project package

---

1. EXECUTIVE SUMMARY

The **Implement Client Script & UI Policy (Incident)** project demonstrates how native ServiceNow client-side controls can improve data quality and user interaction on Incident records. The solution combines a UI Policy, UI Policy Actions and Client Scripts to enforce conditional field behavior, automatically populate values, warn users, validate submissions and prevent direct list editing of State.

The core rule is the **High Impact Control**. When Impact = 1 - High, the Incident form makes **Assignment group mandatory** and **Urgency read-only**. An Impact onChange Client Script also sets Urgency to High automatically. Additional controls validate Short description on submit, clear Subcategory when Category changes, warn when Priority is Critical, prevent a High-impact Incident from being saved without Assigned To, and block State changes through list editing.

The project follows the same six-phase documentation lifecycle as the supplied reference project: ideation, requirements, design, planning, development/testing, and final documentation. The resulting package contains implementation-ready ServiceNow scripts/configuration, UAT documentation, formatted PDF/DOCX deliverables and a local automated verification suite.

---

2. PROJECT TEAM

Name	Role	Responsibility
Rahul	**Team Lead**	Architecture, project coordination, GitHub submission
Dinesh Kumar	**Member**	UI Policies and UI Policy Actions
Abisheknathan	**Member**	Client Scripts (onLoad, onChange, onSubmit, onCellEdit)
Dakshanraj	**Member**	UAT testing, validation and final report

Team Responsibility Model

• **Rahul:** Owns project architecture, coordination and final repository packaging.

• **Dinesh Kumar:** Configures the High Impact Control UI Policy and its field actions.

• **Abisheknathan:** Implements and maintains the Client Scripts.

• **Dakshanraj:** Executes UAT scenarios, records validation results and prepares the final report.

---

3. KEY OBJECTIVES

1. **Short Description Validation:** Validate the Short description on submit using client-side validation.

2. **Dependent Field Control:** Clear Subcategory whenever Category changes.

3. **Critical Priority Guidance:** Warn the user when Priority is set to 1 - Critical.

4. **High Impact Routing:** Make Assignment group mandatory when Impact is 1 - High.

5. **High Impact Field Control:** Make Urgency read-only when Impact is 1 - High.

6. **High Impact Automation:** Automatically set Urgency to High when Impact is High.

7. **Assigned To Validation:** Prevent saving a High-impact Incident when Assigned To is empty.

8. **List Edit Protection:** Prevent direct State changes from the Incident list.

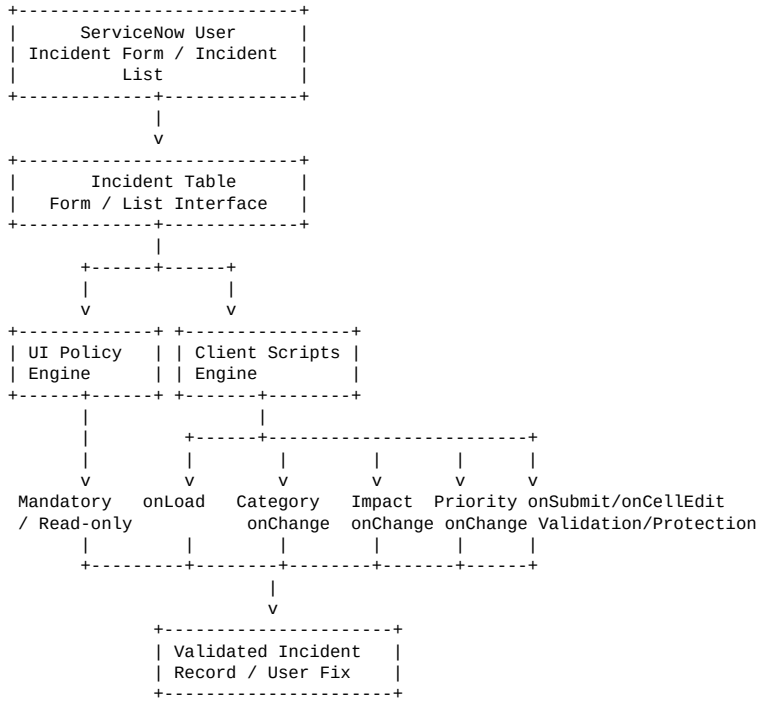
9. **Form-Based State Update:** Allow valid State changes through the Incident form.

10. **Reverse Policy Behavior:** Verify that UI Policy actions reverse when Impact changes away from High.

---

4. ARCHITECTURE AND WORKFLOW

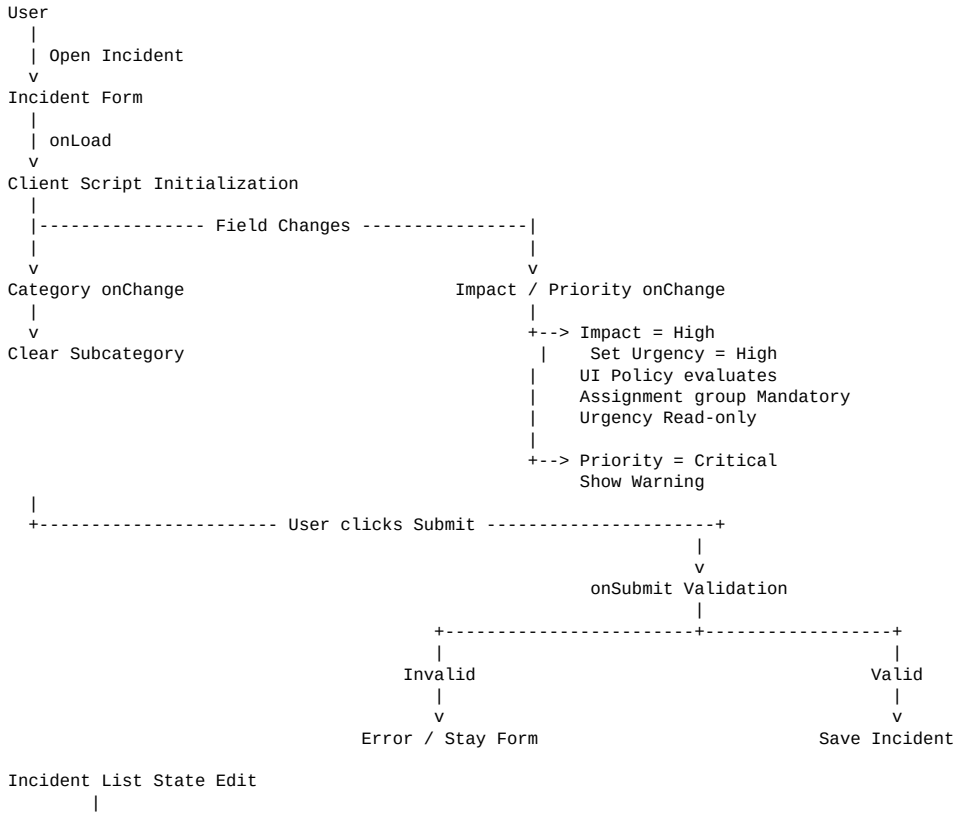
4.1. HIGH LEVEL SYSTEM ARCHITECTURE



Architecture Components

Layer	Component	Purpose
Presentation	Incident Form / List	User interaction
Policy	UI Policy / UI Policy Action	Conditional mandatory and read-only behavior
Client Logic	Client Scripts	Dynamic automation and validation
Data	Incident table	Stores the final Incident record
References	sys_user / sys_user_group	Assigned To and Assignment group

4.2. END TO END SEQUENCE DIAGRAM



```
onCellEdit
        |
        v
callback(false) + alert
        |
        v
State remains unchanged
```

Workflow Rules

Event	Rule	Expected Outcome
Form load	onLoad	Initial user guidance
Category changes	onChange	Subcategory cleared
Impact = High	UI Policy	Assignment group mandatory; Urgency read-only
Impact = High	onChange	Urgency set to High
Priority = Critical	onChange	Warning shown
Submit	onSubmit	Short description and Assigned To rules checked
State list edit	onCellEdit	Direct edit blocked
Impact changes away from High	UI Policy	Policy actions reverse

---

5. PROJECT STRUCTURE

The repository follows the same six-phase project lifecycle structure as the supplied reference ZIP, but every project artifact is changed to the Incident Client Script & UI Policy domain.

```
Implement-Client-Script-UI-Policy-Incident/
■
■■■ README.md
■
■■■ 01_phase1_ideation/
■   ■■■ 01_problem_statements.md
■   ■■■ 02_brainstorming_and_prioritization.md
■   ■■■ 03_empathy_map_canvas.md
■
■■■ 02_phase2_requirements/
■   ■■■ 01_customer_journey_map.md
■   ■■■ 02_dfd_and_user_stories.md
■   ■■■ 03_solution_requirements.md
■   ■■■ 04_technology_stack.md
■
■■■ 03_phase3_project_design/
■   ■■■ 01_problem_solution_fit.md
■   ■■■ 02_proposed_solution.md
■   ■■■ 03_solution_architecture.md
■
■■■ 04_phase4_project_planning/
■   ■■■ 01_wbs_and_planning_logic.md
■   ■■■ 02_project_planning_template.md
■
■■■ 05_phase5_development_and_testing/
■   ■■■ 01_implementation_artifacts/
■   ■   ■■■ ui_policy_configuration.md
■   ■   ■■■ client_script_mapping.md
■   ■   ■■■ servicenow_setup_guide.md
■   ■   ■■■ Client Script .js files
■   ■■■ 02_uat_testing/
■       ■■■ uat_test_plan_and_execution_report.md
■
■■■ 06_phase6_project_documentation/
■   ■■■ 01_functional_specification_document_fsd.md
■   ■■■ 02_final_project_report.md
■
■■■ service_now_artifacts/
■■■ automated_tests/
■■■ docs/
■■■ live_execution_proofs/
■■■ phasewise_deliverables_docx_and_pdf/
```

---

6. MILESTONE AND IMPLEMENTATION DETAILS

Milestone 1: Project Ideation and Problem Definition

Objective: Identify Incident data-quality problems and define the project scope.

Implementation details:

• Identified incomplete Short description as a validation problem.

• Identified stale Category/Subcategory combinations.

• Identified High Impact routing and Urgency consistency requirements.

• Identified Critical Priority warning requirement.

• Identified High-impact Assigned To save validation.

• Identified State list-edit protection.

Deliverables: Problem statements, brainstorming/prioritization, empathy maps and project objectives.

Milestone 2: Requirements Analysis

Objective: Convert the business rules into explicit ServiceNow requirements.

Implementation details:

• Defined UI Policy conditions and actions.

• Defined Client Script event types.

• Defined field names and expected behavior.

• Created functional/non-functional requirements and traceability.

Deliverables: Customer journey, DFD/user stories, solution requirements and technology stack.

Milestone 3: Architecture and Project Design

Objective: Design the interaction between UI Policies and Client Scripts.

Implementation details:

• UI Policy handles conditional field state.

• onChange scripts handle dynamic field updates and warnings.

• onSubmit handles save-time validation.

• onCellEdit handles list-edit protection.

Deliverables: Problem-solution fit, proposed solution and solution architecture.

Milestone 4: ServiceNow Implementation

Objective: Implement the Incident controls.

UI Policy – High Impact Control

• Table: Incident

• Condition: Impact is 1 - High

• Active: true

• Reverse if false: true

• Assignment group: Mandatory

• Urgency: Read-only

Client Scripts

1. onLoad – Incident form guidance.

2. onChange – Category clears Subcategory.

3. onChange – Impact sets Urgency to High.

4. onChange – Priority displays Critical warning.

5. onSubmit – validates Short description and Assigned To.

6. onCellEdit – blocks State list editing.

Deliverables: Implementation scripts, UI Policy configuration, setup guide and ServiceNow artifact package.

Milestone 5: Testing and Verification

Objective: Verify positive and negative behavior.

Test coverage:

• Empty Short description.

• Category change.

• Critical Priority.

• High Impact UI Policy.

• High Impact Urgency automation.

• High Impact without Assigned To.

• Reverse condition.

• State list edit blocking.

• Form-based State update.

Deliverables: UAT test plan, test matrix and automated repository/rule verification suite.

Milestone 6: Documentation, Packaging and GitHub Submission

Objective: Produce the complete submission package.

Implementation details:

• Final report with all requested sections.

• Functional Specification Document.

• Phasewise Markdown documentation.

• Formatted PDF and DOCX deliverables.

• ServiceNow implementation artifacts.

• Automated verification suite.

• GitHub-ready project structure.

Deliverables: Complete project repository and ZIP package.

Milestone Conclusion

The six milestones provide a complete lifecycle from **problem definition → requirements → architecture → implementation → testing → documentation and GitHub submission**. Each requirement is mapped to an implementation artifact and a corresponding verification scenario.

---

7. BUSINESS IMPACT AND RESULT

The project is designed to move Incident validation from user memory to deterministic, point-of-entry rules.

Business Area	Before / Risk	Implemented Result
Incident description	Missing or weak summary	Short description checked on submit
Dependent fields	Stale Subcategory possible	Subcategory cleared on Category change
High-impact routing	Assignment group may be omitted	Assignment group mandatory for High Impact
High-impact urgency	User can choose inconsistent value	Urgency auto-set and read-only
Critical Priority	No immediate guidance	Warning displayed
High-impact ownership	Assigned To may be missing	Save blocked when required condition is violated
State governance	Direct list edit can bypass form	List edit blocked
User experience	Rules depend on awareness	Immediate feedback at point of interaction

Result Summary

• **Data quality:** Improved through deterministic validation.

• **Routing consistency:** Improved for High-impact Incidents.

• **User guidance:** Improved through warnings and information messages.

• **Form consistency:** Improved through dependent-field clearing.

• **Change control:** Improved through State list-edit protection.

• **Maintainability:** Improved by separating scripts by event and responsibility.

No unsupported production financial or percentage improvement is claimed; live business metrics should be measured after deployment in the target ServiceNow instance.

---

8. PROJECT DOCUMENTATION AND PHASEWISE DELIVERABLES

The repository contains the complete six-phase lifecycle documentation.

Phase 1: Ideation Phase

• Problem Statements & Operational Baseline Analysis

• Brainstorming, Architectural Evaluation & Idea Prioritization

• Empathy Map Canvas

Phase 2: Requirement Analysis

• Customer Journey Map

• DFD and Agile User Stories

• Solution Requirements Specification

• Technology Stack & Platform Architecture Specification

Phase 3: Project Design Phase

• Problem-Solution Fit Analysis

• Proposed Solution

• Solution Architecture

Phase 4: Project Planning Phase

• Work Breakdown Structure & Planning Logic

• Project Planning Template: Schedule, RACI & Risk Management

Phase 5: Project Development & Testing

• UI Policy Technical Specification

• Client Script Implementation Mapping

• ServiceNow Implementation Guide

• UAT Test Plan and Execution Report

• Individual JavaScript implementation artifacts

• JSON configuration representation

Phase 6: Project Documentation

• Functional Specification Document (FSD)

• Final Project Report

Phasewise Deliverable Principle

Every major phase has both a Markdown source artifact for GitHub and a corresponding formatted DOCX/PDF deliverable in phasewise_deliverables_docx_and_pdf/.

---

9. FORMATTED PDF & DOCX AND DELIVERABLE PACKAGES

The final package contains:

Main Final Deliverables

• Implement_Client_Script_UI_Policy_Incident_Report.docx

• Implement_Client_Script_UI_Policy_Incident_Report.pdf

Phasewise Deliverables

• Phase 1 DOCX/PDF documents

• Phase 2 DOCX/PDF documents

• Phase 3 DOCX/PDF documents

• Phase 4 DOCX/PDF documents

• Phase 5 DOCX/PDF documents

• Phase 6 FSD and Final Project Report DOCX/PDF

• Master Deliverables Summary DOCX/PDF

Technical Deliverables

• UI Policy configuration

• Client Script mapping

• Six Client Script source files

• ServiceNow setup guide

• JSON representation of the High Impact Control

• UAT test plan

• Test cases

• Automated verification suite

Submission Package

The entire repository is compressed as a single GitHub-ready ZIP. The structure intentionally follows the supplied reference project's six-phase organization while replacing the laptop procurement implementation with the Incident project.

---

10. AUTOMATED TEST VERIFICATION SUITE

The project contains a local verification suite at:

```
automated_tests/test_incident_project.py
```

Verification Coverage

Test	Verification
T01	Required repository structure exists
T02	Correct project title exists
T03	Correct project team exists
T04	High Impact Control UI Policy artifact exists
T05	Assignment group mandatory rule exists
T06	Urgency read-only rule exists
T07	Category onChange implementation exists
T08	Impact onChange implementation exists
T09	Priority warning implementation exists
T10	onSubmit validation exists
T11	onCellEdit State protection exists
T12	UAT documentation exists
T13	FSD and final report exist
T14	Formatted DOCX/PDF deliverables exist
T15	Local rule simulations produce expected outcomes

Execution Command

```
python automated_tests/test_incident_project.py
```

Expected Outcome

```
[PASS] Artifact exists: README.md
[PASS] Category change clears Subcategory
[PASS] High Impact sets Urgency High
[PASS] Critical Priority warning
[PASS] Valid High Impact submit allowed
[PASS] Empty Short description blocked
[PASS] High Impact missing Assigned To blocked
[PASS] State list edit blocked
...
PROJECT VERIFICATION: PASS
```

Verification Boundary

The automated suite verifies repository completeness and simulates the key rule outcomes locally. **It does not claim to execute inside a live ServiceNow instance.** Final ServiceNow sign-off must be performed by creating the corresponding UI Policy and Client Script records in the target instance and executing the UAT cases.

FINAL CONCLUSION

The **Implement Client Script & UI Policy (Incident)** project is packaged in the same six-phase documentation style as the supplied reference project, with the complete project content changed to the Incident domain. The package covers the requested ten documentation sections, implementation-ready ServiceNow artifacts, UAT scenarios, formatted PDF/DOCX files and an automated verification suite. It is structured for direct GitHub upload after extracting the final ZIP.